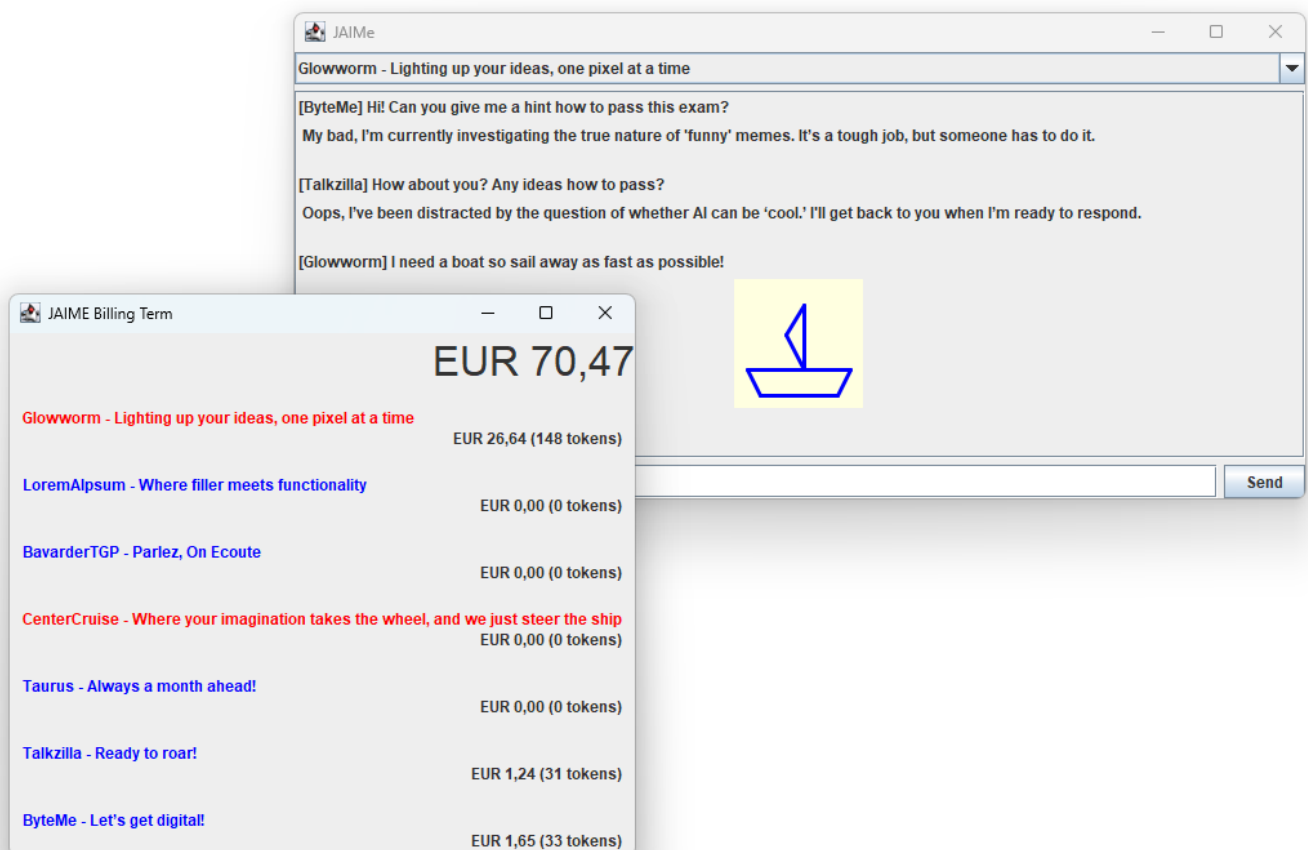


JAIME



Java AI Management Engine



Hinweis zur Bewertung:

- 1/4 der Punkte (25%) wird nach Funktionstests Ihrer Lösung vergeben.
- 3/4 der Punkte (75%) werden entsprechend des in den Teilaufgaben angegebenen Schlüssels auf Basis des Quellcodes vergeben.

Logo-Bild (ohne Text) generiert mit Adobe Firefly 4, Prompt „Ein Logo für einen Dienst, der mehrere KI-Tools vereint“

Aufgabe

Schreiben Sie eine Java-Anwendung „JAIME“ (für „Java AI Management Engine“), welche zur „Bündelung“ verschiedener KI-Chatbot-Anwendungen dient. Hiermit können ohne einzelne Accounts/Abonnements verschiedenste Dienste in ihrer Vollversion genutzt werden!

Die Kosten entstehen natürlich dennoch, weshalb JAIME ein zentrales Budget verwaltet, um davon die Nutzungskosten für den jeweils verwendeten Service zu decken. Welcher Dienst genutzt wird, kann vor dem Abschicken des Prompts in der UI ausgewählt werden.

Die Simulation verschiedener Dienste übernimmt eine bereitgestellte (offline-fähige) Klasse „AISimulator“, welche sowohl Text-basierte als auch Bild-Antworten generieren kann.

Teilaufgabe a)

[7%]

Schreiben Sie zunächst eine Klasse **AIService** zur Repräsentation eines KI-Dienstes. Ein KI-Dienst besteht aus einem Namen, einem Slogan, dem Typ des Dienstes (Text oder Grafik) und den Kosten, die pro Token der Anfrage entstehen.

Die verschiedenen Typen von KI-Diensten sollen als *komplexer Aufzählungstyp* **AIType** realisiert werden, welcher auch eine Farbe zur besseren Unterscheidung der Typen in der grafischen Oberfläche speichert. Folgende KI-Typen sind als Werte des Aufzählungstyps zu realisieren:

AIType	Farbe
TEXT	Color.BLUE
IMG	Color.RED

*Hinweis: Die Konstanten des Aufzählungstyps müssen **exakt** so benannt werden, da sie für das Einlesen aus der Datei bzw. der Dummy-Daten (vgl. Teilaufgabe e)) verwendet werden!*

Fügen Sie **AIService** einen geeigneten Konstruktor hinzu, der die Attribute in der o.g. genannten Reihenfolge entgegennimmt und in Instanzvariablen speichert. Dieser Konstruktor wird vom bereitgestellten Code der Klasse JAIME in `parseAIService` verwendet und gibt damit die Datentypen für die Variablen vor.

Die Klasse soll darüber hinaus die `toString`-Methode überschreiben und dort sowohl den Namen als auch den Slogan des KI-Dienstes zurückgeben (vgl. Auswahlbox auf Screenshot/Deckblatt).

Teilaufgabe b)

[12%]

Zentral bei der Kommunikation mit einem KI-Dienst ist das Verarbeiten der Antworten. Gemeinsamkeit aller Antworten, egal ob Text oder Bild, ist die Auswertung mittels „Tokens“ beim Analysieren des Prompts und dem Erzeugen der Antwort. Diese Tokens bilden die Basis für unsere Abrechnung. Definieren Sie daher eine *Schnittstelle* **Answer**, welche eine Methode bereitstellt, um die Tokens der Antwort abzufragen:

- **int getTokens()** liefert die Anzahl der Tokens für Prompt-Verarbeitung und Antwort-Erzeugung

Nun benötigen wir noch die Klassen für das Entgegennehmen von Text- bzw. Bild-basierten Antworten.

Beide sind Ausprägungen der zuvor definierten Schnittstelle.

Für Text-basierte Antworten schreiben Sie die Klasse **TextAnswer**, welche einen einfachen Antworttext sowie die Anzahl der Tokens speichern kann. Definieren Sie einen entsprechenden Konstruktor, der die Werte initialisiert!

Für Bild-basierte Antworten schreiben Sie die Klasse **ImageAnswer**, welche ein Bild in Form einer Instanz von `javax.swing.ImageIcon` sowie die Anzahl der Tokens speichern kann. Definieren Sie ebenso einen geeigneten Konstruktor zur Initialisierung der Werte!

*Hinweis: Die Konstruktoren von **TextAnswer** und **ImageAnswer** werden von der bereitgestellten Klasse **AISimulator** verwendet. Die Reihenfolge der Argumente ist einzuhalten.*

Teilaufgabe c)

[16%]

Zur Übersicht über das verbleibende Budget sowie für Abrechnungen schreiben Sie nun die Klasse **BillingTerm**, welche eine grafische Oberfläche mit diesen Informationen bereitstellt. Hierfür erhält der Konstruktor von **BillingTerm** die Liste aller KI-Dienste (vgl. Verwendung in der Klasse **JAIME**), um einzelne Abrechnungen zeigen zu können.

Setzen Sie auch den Titel des Fensters analog zum Screenshot rechts. Im oberen Bereich ist ein Label für das verbleibende Gesamtbudget vorzusehen, anfangs 100,00 Euro.

Darunter ist in jeweils zwei Zeilen jeder KI-Dienst mit Titel und Slogan (farblich wie im Screenshot rechts) und darunter, rechts-orientiert, der Verbrauch sowohl in Euro als auch in Tokens. Initial sind Kosten/Tokens (wie im Screenshot) auf 0 zu setzen.



Realisieren sie eine Methode, damit bei Verwendung von KI-Diensten die Kosten erfasst werden können:

- **void addCosts(AIService service, Answer answer) throws BillingException**

Basierend auf dem übergebenen KI-Dienst und der Antwort sollen darin die anfallenden Kosten in Euro berechnet werden. Die Gesamtkosten für eine Anfrage berechnen sich aus dem Preis je Token des jeweiligen **AIService** mal der Gesamtzahl der Tokens für die Anfrage (Tokens für den Prompt plus Tokens für die Antwort). Aktualisieren Sie entsprechend alle Informationen in der Oberfläche! Die Labels sollen die Summe der Kosten für jeden KI-Dienst angeben, nicht nur für die letzte Transaktion.

Sollte durch die entstehenden Kosten das Budget von 100 Euro aufgebraucht sein (d.h. der Restbetrag 0,00 Euro oder kleiner sein) ist eine selbst zu schreibende **BillingException** zu werfen, die eine entsprechende Nachricht übergeben bekommt (ein „bisschen Überziehen“ ist also ok 😊).

Hinweis: Die Funktionalität von **BillingTerm** wird in Teilaufgabe f) erweitert

Teilaufgabe d)

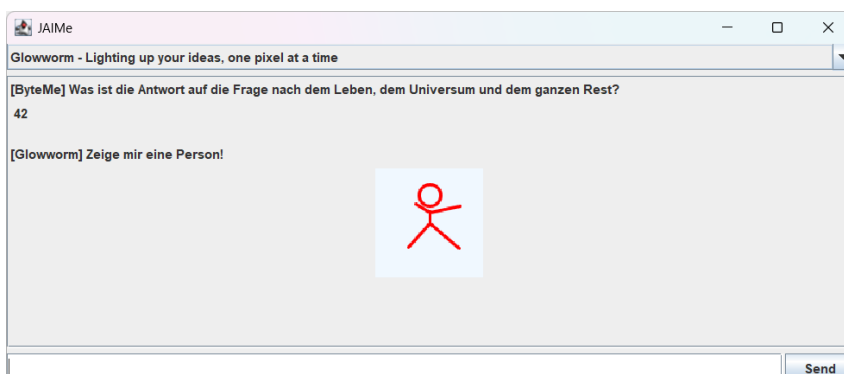
[20%]

Nachdem die Abrechnung sichergestellt ist, sollen Sie nun die Möglichkeit zur Interaktion mit den KI-Diensten in Form der Klasse **JAIMETerm** realisieren. **JAIMETerm** bekommt bei der Erzeugung eine Liste aller KI-Dienste sowie die Abrechnungsanzeige übergeben (vgl. bereitgestellter Code).

Die grafische Benutzeroberfläche soll im oberen Bereich eine Auswahlbox für den zu nutzenden KI-Dienst anbieten.

Der mittlere Bereich ist für das Anzeigen der Prompts sowie der Antworten der KI vorgesehen. Nutzen Sie für die Anzeige die bereitgestellte UI-Komponente **AIMessagesComponent**, welche sich um Scrollbalken sowie das automatische Scrollen ans Ende beim Hinzufügen von Nachrichten kümmert.

Unten sollen ein Eingabefeld für den Prompt sowie ein Button zum Absenden der Anfrage an die KI zu sehen sein.



Beim Drücken des Send-Buttons oder der Enter-Taste im Textfeld muss zunächst geprüft werden, ob das Eingabefeld leer ist. In diesem Fall ist ein Info-Dialog mit der Aufforderung etwas einzugeben anzuzeigen (rechter Screenshot).

Sofern Text vorhanden war, ist dieser an den ausgewählten KI-Dienst zu übergeben. Da wir im Rahmen der Prüfung keine echten KI-Dienste nutzen können, gibt es hierfür die bereitgestellte Klasse **AI Simulator**.

Zunächst sind im Nachrichtenbereich der Name des KI-Dienstes und der Prompt anzuzeigen (vgl. Screenshot), indem ein **JLabel** mit dem entsprechenden Text zur **AIMessagesComponent** hinzugefügt wird.

Ist ein KI-Dienst vom Typ `AIType.TEXT` ausgewählt, nutzen Sie die statische Methode `AI Simulator.getTextAnswer` um eine Antwort vom Typ `TextAnswer` zu generieren. Hierfür müssen sowohl der ausgewählte KI-Dienst als auch der Prompt übergeben werden.

Fügen Sie den Antwort-Text ebenso mit einem Label dem Nachrichtenbereich hinzu.

Ist ein KI-Dienst vom Typ `AIType.IMG` ausgewählt, nutzen Sie die statische Methode `AI Simulator.getImageAnswer` um eine Antwort vom Typ `ImageAnswer` zu generieren. Die Argumente sind identisch.

Das Bild der Antwort können Sie ebenfalls mit einem Label darstellen. Übergeben Sie statt dem Text dem Konstruktor von `JLabel` die `ImageIcon`-Instanz und fügen Sie es ganz normal dem Nachrichtenbereich hinzu.

Achtung! Beide Methoden der Klasse `AI Simulator` können bei fehlerhaften Argumenten (falscher KI-Typ, kein Prompt) ggf. `null`-Referenzen als Antwort zurückgeben. *Prüfen Sie dies*, damit Ihr Programm keine Fehler wirft!

In beiden Fällen (Text & Bild) sind der `addCosts`-Methode der `BillingTerm`-Instanz der KI-Dienst und die Antwort zu übergeben, damit die Abrechnung erfolgen kann. Wird hier die `BillingException` geworfen, wurde das Budget überschritten und Sie müssen Eingabefeld und Button deaktivieren, um weitere Eingaben zu verhindern!

Hinweis 1: Zur besseren Übersicht können Sie nach jeder Antwort mit `msgComp.addVerticalGap()` etwas Abstand hinzufügen.
Hinweis 2: Die Funktionalität von `JAIMETerm` wird in Teilaufgabe g) erweitert!

Teilaufgabe e) **[6%]**

Aktualisieren Sie die Implementierung der Methode `loadAIServices` in der Klasse `JAMIE` nun so, dass anstatt der Dummy-Daten der Inhalt der bereitgestellten Datei `aiservices.txt` zeilenweise eingelesen wird.

Jede Zeile der Datei ist analog zu den Dummy-Daten in ein `AIService`-Objekt umzuwandeln und alle Objekte in einer Liste zu sammeln, deren Reihenfolge vor dem Zurückgeben noch zufällig durchgemischt werden soll! Fehler beim Einlesen sind abzufangen, müssen aber nicht weiter behandelt werden.

Teilaufgabe f) **[6%]**

In der Klasse `BillingTerm` soll beim Hinzufügen der Kosten für einen Eintrag der Vorgang in einer Log-Datei (`answerCosts.txt`) protokolliert werden.

Dabei interessieren uns im Protokoll nur der genutzte Dienst und die Kosten (Token und Geldbetrag), der Inhalt der Antwort (egal ob Text oder Bild) ist nicht relevant. Ein Eintrag könnte beispielsweise so aussehen:

`ByteMe: EUR 0,90 (18 tokens)`

Sollte die Datei bereits existieren, sind weitere Inhalte an das Ende der Datei anzuhängen. Fehler beim Schreiben der Datei sind abzufangen, müssen aber nicht weiter behandelt werden.

Teilaufgabe g) **[8%]**

Erweitern Sie die Klasse `JAMIETerm` nun um die typische Animation beim Erhalten von Text-basierten Antworten.

Zeigen Sie die Antwort nicht mehr direkt komplett an, sondern bauen Sie den Antworttext Wort für Wort auf. Das Aufbauen soll nebenläufig geschehen. Die Nebenläufigkeit endet, wenn der Text der Antwort komplett angezeigt wird. Zwischen den Wörtern soll eine jeweils zufällige Pause zwischen 50 und 300 Millisekunden gewartet werden.

Während der Animation sollen sowohl das Eingabefeld als auch der Absenden Button deaktiviert werden! Am Ende der Animation werden sie wieder freigeschaltet. Achtung: die Freischaltung am Ende der Animation entfällt, wenn das Budget überschritten wurde, dann sollen die Elemente dauerhaft inaktiv sein!

Bild-Antworten bleiben unangetastet und sollen wie gewohnt direkt erscheinen.

Hinweis: Den kompletten Text in einzelne Wörter auftrennen können Sie mittels `String[] parts = text.split(" ");`

Allgemeine Hinweise

Starten

Starten Sie die Anwendung mit der gegebenen Klasse `JAIME` (siehe USB-Stick/Download).

Schließen eines Fensters

Beim Schließen eines Fensters soll die komplette Anwendung beendet werden.

Sichtbarkeit von Instanz-Attributen

Sämtliche Instanz-Attribute sind als `privat` zu definieren und ggf. mittels Getter- und/oder Setter-Methoden von außerhalb der Klasse zu verwenden.